

day 11

📌 01. Asynchronous JavaScript & Callbacks

💡 المفهوم الأساسي جافاسكريبت هي **Single-threaded Language** (تتفّذ أمرًا واحدًا في الوقت نفسه).
لمعالجة العمليات الثقيلة (مثل طلبات البيانات أو التايمر) دون تجميد الشاشة، نستخدم **Asynchronous Programming**.

1. What is a Callback Function?

دالة تُمرر كـ **Parameter** لدالة أخرى، ويتم استدعاؤها بعد انتهاء العملية غير المتزامنة.

JavaScript

```
function fetchData(callback) {  
  setTimeout(() => {  
    const data = "hello";  
    callback(data); // عند تجهيز البيانات callback استدعاء الـ  
  }, 5000);  
}  
  
fetchData((result) => {  
  console.log(result); // بعد 5 ثوان "hello" يطبع  
});
```

2. Callback Hell (Pyramid of Doom)

عندما نعتمد على بيانات عمليات متتالية (مثل: جلب المستخدم ➡ جلب بوساته ➡ جلب التعليقات)، تداخل الكولباكس يجعل الكود صعب القراءة والتتبع الصيانة.

JavaScript

```
function fetchData1(callback) {  
  setTimeout(() => callback("returned data 1 "), 2000);  
}  
  
function fetchData2(data1, callback) {  
  setTimeout(() => callback(data1 + "data2 "), 1000);  
}  
  
function fetchData3(data2, callback) {  
  setTimeout(() => callback(data2 + "data3"), 1000);  
}
```

```

}

// ⚠️ Callback Hell Structure
fetchData1((data1) => {
  fetchData2(data1, (data2) => {
    fetchData3(data2, (data3) => {
      console.log(data3); // Output: "returned data 1 data2 data3"
    });
  });
});
});

```

📌 02. Promises (promise.js)

💡 الحل الأنيق لـ **Callback Hell Promise** هو **Built-in Object** في JS يمثل قيمة ستكون متاحة مستقبلاً (إما بنجاح **Resolve** أو بفشل **Reject**).

🚦 Promise States (حالات الـ Promise)

1. **Pending**: الحالة الأولية (قيد الانتظار).
2. **Fulfilled (Resolved)**: نجحت العملية وتم إرجاع البيانات.
3. **Rejected**: فشلت العملية وتم إرجاع الخطأ.

JavaScript

```

let myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    // resolve("data received"); // حالة النجاح
    reject("error"); // حالة الفشل
  }, 1000);
});

myPromise
  .then((result) => console.log(result)) // Resolve يُنفذ عند الـ
  .catch((error) => console.log(error)); // Reject يُنفذ عند الـ

```

🔗 Promise Chaining

حل مشكلة الكولباك هيل عن طريق ربط السلسلة بـ `.then()` المتتالية.

JavaScript

```

function fetchData1() {
  return new Promise((res, rej) => {

```

```

        setTimeout(() => res("data1 "), 2000);
    });
}

function fetchData2(data1) {
    return new Promise((res, rej) => {
        setTimeout(() => res(data1 + "data2 "), 1000);
    });
}

function fetchData3(data2) {
    return new Promise((res, rej) => {
        setTimeout(() => res(data2 + "data3"), 1000);
    });
}

// 🟢 Clean Promise Chaining
fetchData1()
    .then(data1 => fetchData2(data1))
    .then(data2 => fetchData3(data2))
    .then(data3 => console.log(data3))
    .catch(error => console.log(error));

```

📌 03. Async / Await

✨ أحدث وأفضل طريقة (Syntactic Sugar) تجعل الكود غير المتزامن يكتب بأسلوب يبدو وكأنه متزامن (Synchronous)، مما يسهل القراءة والتتبع باستخدام `try...catch` ✓

JavaScript

```

async function fetchAllData() {
    try {
        const data1 = await fetchData1(); // ينتظر 2 ثانية
        const data2 = await fetchData2(data1); // ينتظر 1 ثانية
        const data3 = await fetchData3(data2); // ينتظر 1 ثانية
        console.log(data3);
    } catch (error) {
        console.log("Caught Error:", error);
    }
}

fetchAllData();

```

📌 04. Execution Order & Microtasks (Interview Classic)

عند تنفيذ الكود، الـ Call Executor الخاص بالـ Promise يُنفذ **Synchronously** فوراً، ولكن ما بداخل .then() يذهب إلى **Microtask Queue** ويُنفذ بعد الانتهاء من الـ Synchronous Code.

JavaScript

```
console.log("1");

const promise = new Promise((resolve) => {
  console.log("2"); // 🟢 Promise ينفذ فوراً عند إنشاء الـ
  resolve("resolved");
});

promise.then((res) => {
  console.log("3"); // 🟡 Microtask Queue يذهب للـ
  console.log("5");
});

console.log("4");

// 🏷️ Output Order:
// 1
// 2
// 4
// 3
// 5
```

📌 05. Web Storage API (localStorage vs sessionStorage)

مميزات في المتصفح لتخزين البيانات على جهاز المستخدم بصيغة (Key-Value) بنصوص فقط (Strings).

JavaScript

```
// 🟢 LocalStorage: تُحفظ البيانات حتى بعد إغلاق المتصفح أو إعادة تشغيل الجهاز
localStorage.setItem("name", "ahmed");
localStorage.setItem("course", "JS");

console.log(localStorage.getItem("course")); // 🟢 "JS"
localStorage.removeItem("course");           // حذف عنصر محدد
// localStorage.clear();                     // مسح كل شيء

// 🟡 SessionStorage: تُحفظ البيانات فقط طوال مدة فتح التاب الحالي (تُمحى فور إغلاق التاب)
sessionStorage.setItem("name", "ahmed");
```

```
sessionStorage.setItem("course", "JS");

console.log(sessionStorage.getItem("course"));
sessionStorage.removeItem("course");
```

جدول المقارنة الشامل (Web Storage) 📋

Feature	localStorage	sessionStorage	Cookies
Expiration	لا تنتهي إلا بحذفها يدويًا	تنتهي بمجرد إغلاق التاب (Tab) (Close)	محددة بـ Expiration Date
Capacity	~5MB - 10MB	~5MB	~4KB
Server Transfer	لا تُبعث مع الـ HTTP Requests	لا تُبعث مع الـ HTTP Requests	تُبعث تلقائيًا مع كل HTTP Request

❓ أسئلة إنترفيو متوقعة (Interview Q&A)

❓ Q1: What is the main difference between Microtask Queue and Macrotask Queue? الإجابة:

- **Microtask Queue:** و (.then/catch/finally) Promises تحتوي على الـ process.nextTick . أخرى task لها الأولوية المطلقة وتُفرغ بالكامل قبل الانتقال لأي .
- **Macrotask Queue:** تحتوي على I/O operations , setTimeout , setInterval , و الـ Reject الأخطاء في حالة حدوث

❓ Q2: Why do we use try...catch with async/await ؟ الإجابة: لأن async/await لا لالتقاط try...catch فيستخدم ، Promise chaining مدمجة بشكل مباشر كـ .catch() تحتوي على دالة Promise في الـ Reject الأخطاء في حالة حدوث

❓ Q3: What happens if you pass an Object to localStorage.setItem() ؟ الإجابة: ولحفظه بشكل "[object Object]" فيتخزن كـ .toString() تحوله جافاسكريبت تلقائيًا إلى نص باستدعاء JSON.stringify(obj) عند التخزين و JSON.parse() عند القراءة . صحيح يجب استخدام